

Awake in Omelas / KIOS Handoff Bundle - Review

2026-09-13

1. Architecture and responsibility of each repository

awake-in-omelas - the canonical, governed project repository. Its own AI-REPOSITORY-METADATA.md and RRA-0001 report (artifacts/governance/repository-responsibility/report.adoc) describe it as the repo holding domain canon (canon/), templates, GAE (Governed Artifact Execution) evidence and prompts (docs/gae/, docs/engineering/governed-artifact-execution/), operations/bootstrap docs, and its own original MCP launcher scripts (scripts/mcp/run-mcp.bash, run-readonly.bash, run-readwrite.bash) that run the “Alpha” filesystem container image mcp/filesystem:latest. PROJECT.adoc documents the KIOS framework itself (Knowledge - Integrity - Order - Story) - a provenance-first, Git-native model for canonical knowledge, currently housed inside this repo. RRA-0001 explicitly states the repo’s job is to determine which of its artifacts are project-specific vs. reusable infrastructure that should move elsewhere - i.e., the repo already anticipates its own decomposition.

governed-mcp-runtime - a separate, dedicated repository (git@github.com:rirl/governed-mcp-runtime.git) for the MCP/MCPO tooling and runtime. Per its README.adoc, it’s “an R&D refactor that preserves the canonical launcher while making runtime execution replaceable”: the default runtime stays the Alpha filesystem behavior (scripts/mcp/runtimes/filesystem.bash), and MCPO is opt-in via MCP_RUNTIME=mcpo (scripts/mcp/runtimes/mcpo.bash). It owns the Dockerfile, dependency pinning, image-selection logic, health/governance libraries, and the tests validating them.

awake-in-omelas-program-control - a control-only repository (its README states explicitly: “*It is a control repository, not an implementation repository*”). It records proposed sequencing, milestones, architectural decisions, and accepted baselines for the Omelas/KIOS/GAE/runtime separation program. It does not implement anything itself.

2. What changed during recovery vs. MCP runtime hardening (kept separate)

Recovery (docs/operations/mcp-runtime-pinning-checkpoint.md, baseline checkpoint-recover-091406):

- Recreated the deleted open-webui-tf Terraform-managed container (plan: 1 to add, 0 to change, 0 to destroy), verified healthy via GET /health.
- Recovered the Omelas-MCP-RW container on omelas/mcpo-filesystem:beta, confirmed OpenAPI identity (secure-filesystem-server v0.2.0) and runtime identity uid=1001 gid=1001.
- This established the known-good runtime baseline from commit

0243f43 before any hardening changes were made.

Runtime hardening (separate, subsequent commits):

- First pass: pinned mutable build inputs (base image digest, package versions) - commit e716709 (“pin runtime dependencies and record recovery validation”).
- Second pass: split build vs. runtime image selection - commits 4d3f8eb / b776e55 (“separate build and runtime image selection”).
- Registry-durability validation - commit 40eec4b (“record registry recovery validation”), the current HEAD of feature/runtime-v0.2.

Each doc explicitly states it does **not** perform KIOS/Omelas consolidation or architectural refactoring - this separation is stated as a boundary condition in all three documents, not just an interpretation.

3. Dependency-pinning and image-selection changes

From docker/mcp/Dockerfile.mcpo-filesystem and docker/mcp/constraints.txt:

- Base image pinned to node:22.23.2-bookworm-slim@sha256:4d676821...
- Python deps pinned via constraints: mcpo==0.0.20, mcp==1.28.1.
- npm package pinned: @modelcontextprotocol/server-filesystem@2026.8.31.

Image-selection split (docs/operations/mcp-runtime-image-selection.md, implemented in scripts/mcp/runtimes/mcpo.bash and scripts/mcp/build-mcpo-filesystem.bash):

- MCP_MCP0_BUILD_IMAGE - mutable build destination tag (default omelas/mcpo-filesystem:beta).
- MCP_MCP0_RUNTIME_IMAGE - immutable runtime reference, defaulting in code today to omelas/mcpo-filesystem@sha256:2a55c88be6... (confirmed in both scripts/mcp/runtimes/mcpo.bash and the committed test tests/mcp/image-selection.bash).
- Legacy MCP_MCP0_IMAGE retained for both paths with deprecation warnings, for compatibility.
- Test coverage in tests/mcp/image-selection.bash passes for: default digest resolution, explicit override, legacy-var compatibility, and build-script support for both variables.
- A separate, documented correction: MCP_API_KEY had been mistakenly sourced from OPENAI_API_KEY; this was corrected to an independently generated local token, with no script interface change.

4. The four-phase GHCR registry recovery validation

Documented in full in docs/operations/mcp-runtime-registry-recovery-validation.md:

- **Phase 1 - authenticated remote availability:** confirmed GHCR (ghcr.io/r1rl/governed-mcp-runtime@sha256:2a55c88...) resolves the exact validated digest while the local image still existed. **PASS.**
- **Phase 2 - local image loss simulation:** removed local tags (omelas/mcpo-filesystem:pinned-test, the GHCR tag) and confirmed

the underlying image object was absent from the local Docker store, while a separate mutable :beta artifact remained untouched. **PASS.**

- **Phase 3 - authenticated recovery by immutable digest:** pulled the image back from GHCR by digest and confirmed the recovered image ID matched the original exactly. **PASS.**
- **Phase 4 - governed runtime recovery:** used MCP_MCP0_RUNTIME_IMAGE to point at the recovered GHCR digest and ran resume-mcp0.bash readwrite; governance validation (branch = feature/openwebui-mcp-filesystem, access mode, derived container name Omelas-MCP-RW) succeeded, and authenticated OpenAPI checks matched the known-good service identity. **PASS.**

Important nuance: the GHCR digest was **explicitly injected via override** for this test (MCP_MCP0_RUNTIME_IMAGE=ghcr.io/...), not made the new hard-coded default - the scripts and committed test still default to the local-namespace digest reference (omelas/mcp0-filesystem@sha256:...). GHCR publication is proven durable, but not yet wired as the runtime's default source.

5. Proven vs. deferred

Proven (explicitly stated “Result: PASS” / “validated end to end”):

- Reproducible pinned build matches the known-good runtime behaviorally.
- Build vs. runtime image identity separation works and is tested.
- The exact validated artifact (same digest) can be published to a private, authenticated GHCR repo without rebuilding.
- That exact artifact can be recovered after simulated local loss, using only authenticated registry pull by digest.
- The recovered artifact runs successfully through the real governed resume path (resume-mcp0.bash readwrite), with matching runtime identity, port binding, and OpenAPI signature.
- Credential provenance for MCP_API_KEY was clarified and corrected.

Explicitly deferred (per the “Current boundary” / “Deferred” sections in both the image-selection and registry-recovery docs):

- Publishing/setting GHCR as the runtime's *default* image source (still an explicit override today).
- Removing the deprecated MCP_MCP0_IMAGE variable.
- Renaming MCP_API_KEY.
- Pinning Debian packages/snapshot dates, uvicorn / fastapi pinning, Python patch-version pinning.
- OCI source/revision labels, automated CI publication to GHCR, SBOM generation/enforcement, image signing, centralized registry (Nexus/Artifactory) ingress, approved-image policy enforcement.
- Any KIOS/Omelas architectural consolidation or extraction work - repeated verbatim as out of scope in every runtime doc.

6. Current KIOS/Omelas consolidation state

Per awake-in-omelas-program-control/baselines/2026-09-11-consolidation-control-state.adoc (status: *Validated Program-Control Checkpoint*):

- awake-in-omelas working branch feature/openwebui-mcp-filesystem is at a478481, clean and synced with origin. GAE-0001 through

- GAE-0007 are complete; no GAE-0008 has been adopted.
- development is at f6a1fba, clean and synced, but **development...feature/openwebui-mcp-filesystem = 0 49** - development has no commits the feature branch lacks, but the feature branch has 49 commits development doesn't have. So the branches are not diverged, but development is not yet the intended parking baseline because it doesn't contain the completed GAE/MCP/governance work.
- "Major KIOS / Omelas separation work has not begun." Phase A (Responsibility Inventory) of the planned separation is explicitly **not yet active**.
- The program is positioned *between* closing the feature branch and establishing development as the pre-separation parking baseline - i.e., still in the pre-Phase-A control window.

7. Relationship between feature/openwebui-mcp-filesystem and development

Confirmed by the Git metadata diff (## Diff against development, 79 files / 7,464 insertions, 8 deletions) and the baseline doc's stated 0 49 comparison:

- feature/openwebui-mcp-filesystem is **strictly ahead** of development by 49 commits - a fast-forward relationship, not a divergence. development contains nothing the feature branch lacks.
- The feature branch carries all completed GAE-0001-0007 work, the RRA-0001 governance/responsibility assessment, breakpoint-beta MCPO prototype artifacts, validation records, and canon additions that development does not yet have.
- Per the roadmap (integration-separation-control-timeline.adoc), the intended integration method is to merge (preferred over rebase, absent a concrete reason) this branch into development, then validate and record the result as the new parking baseline - but that integration has **not yet occurred**; both the roadmap and the 2026-09-11 baseline agree the merge is still a pending "DECISION"/"NEXT" item, not a completed one.

8. Next safe consolidation checkpoint

The repositories converge on the same next step from two independent angles - the program-control baseline and the registry-recovery doc's own "next program-control step":

1. Reconcile live Git state of the awake-in-omelas and awake-in-omelas-program-control repositories.
2. Decide and record the integration method for feature/openwebui-mcp-filesystem → development (merge preferred).
3. Perform the integration and post-merge validation.
4. Record the resulting development HEAD as the validated parking baseline.
5. Only then create a new, explicitly-named architecture/separation branch from that admitted baseline and begin Phase A (Responsibility Inventory).

Both the program-control baseline doc and the runtime docs are explicit and consistent that **no extraction, boundary design, or architectural refactoring should begin before this integration-and-parking sequence is complete** - the MCP runtime hardening work is described as having cleared its own blockers ("no remaining

blocking MCP runtime dependency or artifact-durability issues”) but that clearance is scoped only to *resuming* the consolidation path, not to skipping the parking-baseline step itself.